

DevSecOps 4-5-Container-Hardening

Task 1 - Introduction

This module has taught you the basics of Docker and the potential vulnerabilities associated with containers. This room will teach you the various ways of securing your containers. Learning Objectives:

- Secure the Docker daemon to prevent unauthorised interaction.
- Correctly assign privileges (capabilities) to containers.
- Prevent faulty or malicious containers from exhausting a system.
- Utilise security features such as Seccomp and AppArmor to define how your containers will behave with the operating system.
- Practice good hygiene by reviewing Docker images for vulnerabilities - Implement frameworks and tooling that reviews your code for vulnerabilities.

Task 2 - Protecting the Docker Daemon

The Docker daemon manages container operations and image registries, often being controlled remotely within CI/CD pipelines to build, test, and deploy applications.

Exposing the Docker daemon to networks—a common practice in cloud and CI/CD environments—creates severe risks, as attackers who access it can run malicious containers or steal sensitive data unless strong authentication and communication controls are enforced.

SSH: Developers can use SSH authentication and Docker contexts (profile configurations) to securely connect to and switch management between different remote Docker host environments.

Note: You must have SSH access to the remote device, and your user account on the remote device must have permission to execute Docker commands.

As the developer, you will need to create the Docker context on your device. Please see the code snippet below to create a context within Docker. Creating a new Docker context

```
client@thm:~# docker context create
--docker host=ssh://myuser@remotehost
--description="Development Environment"
development-environment-host
```

Successfully created context "development-environment-host"

Once this has been completed, you can switch to this context, where all Docker-related commands will now be executed on the remote host. Using our newly created Docker context

```
cmnatic@thm:~# docker context use development-environment-host
```

Current context is now "development-environment-host"

To exit this context and, for example, use your own Docker engine, you can revert to "default" via `docker context use default`.

Note: This is not entirely secure. For example, a weak SSH password can lead to an attacker being able to authenticate. Strong password hygiene is strongly recommended. Some tips for a strong password have been included below:

- A high amount of characters (i.e. 12-22+)
- Special characters such as !, @, #, \$
- Capital letters and numbers placed sporadically throughout (i.e. sUp3rseCreT!PaSSw0rd!)

Docker contexts allow you to interact with the Docker daemon directly over SSH, which is a secure and encrypted way of communication.

TLS Encryption: The Docker daemon can be accessed remotely over HTTPS, using TLS encryption to secure transmitted data and restrict remote command

execution strictly to authenticated clients with valid signed certificates.

Note: Although TLS certificate creation involves environment-specific management factors like expiry dates and encryption strength, once generated, these certificates are supplied to configure the host Docker daemon to run in TLS mode. On the host (server) that you are issuing the commands from: Running Docker in TLS mode

```
server@thm:~# dockerd --tlsverify --tlscacert=myca.pem --tlscert=myserver-cert.pem --tlskey=myserver-key.pem -H=0.0.0.0:2376
```

On the host (client) that you are issuing the commands from: Telling Docker (local) to authenticate using TLS

```
client@thm:~# docker --tlsverify --tlscacert=myca.pem --tlscert=client-cert.pem --tlskey=client-key.pem -H=SERVERIP:2376 info
```

Note: It is important to remember that this is not guaranteed to be secure. For example, anyone with a valid certificate and private key can be a "trusted" device. I have explained the arguments used in generating a TLS certificate and key in the table below:

Argument	Description
<code>--tlscacert</code>	This argument specifies the certificate of the certificate authority. A certificate authority is a trusted entity that issues the certificates used to identify devices.
<code>--tlscert</code>	This argument specifies the certificate that is used to identify the device.
<code>--tlskey</code>	This argument specifies the private key that is used to decrypt the communication sent to the device.

Answer the questions below

What would the command be if we wanted to create a Docker profile?

`docker context create`

What would the command be if we wanted to switch to a Docker profile?

`docker context use`

Task 3 - Implementing Control Groups

Control Groups (cgroups) are a Linux kernel feature that restricts, prioritizes, and tracks system resource usage (such as RAM and CPU) for processes to enhance overall system stability.

In Docker, cgroups provide a vital secondary defense by limiting per-container resource consumption, preventing faulty or malicious containers from exhausting host resources and causing system crashes. The switches used to specify the limit of resources a container can use have been provided in the table below:

Type of Resource	Argument	Example
CPU	<code>--cpus</code> (in core count)	<code>docker run -it --cpus="1" mycontainer</code>
Memory	<code>--memory</code> (in k, m, g for kilobytes, megabytes or gigabytes)	<code>docker run -it --memory="20m" mycontainer</code>

You can also update this setting once the container is running. To do so, use the `docker update` command, the new memory value, and the container name. For example: `docker update --memory="40m" mycontainer`. You can use the `docker inspect containername` command to view information about a container (including the resource limits set). If a resource limit is set to **0**, this means that no resource limits have been set. Using Docker inspect to list the resource limits set for a container.

```
cmnatic@thm:~# docker inspect mycontainer
--cropped for brevity--
  "Memory": 0,
  "CpuQuota": 0,
  "CpuRealtimePeriod": 0,
```

```
"CpuRealtimeRuntime": 0,  
"CpusetCpus": "",  
"CpusetMems": "",  
"CpuCount": 0,  
"CpuPercent": 0,  
--cropped for brevity--
```

Docker uses namespaces to create isolated environments. For example, namespaces are a way of performing different actions without affecting other processes.

Answer the questions below

What argument would we provide when running a Docker container to enforce how many CPU cores the container can utilise?

--cpus

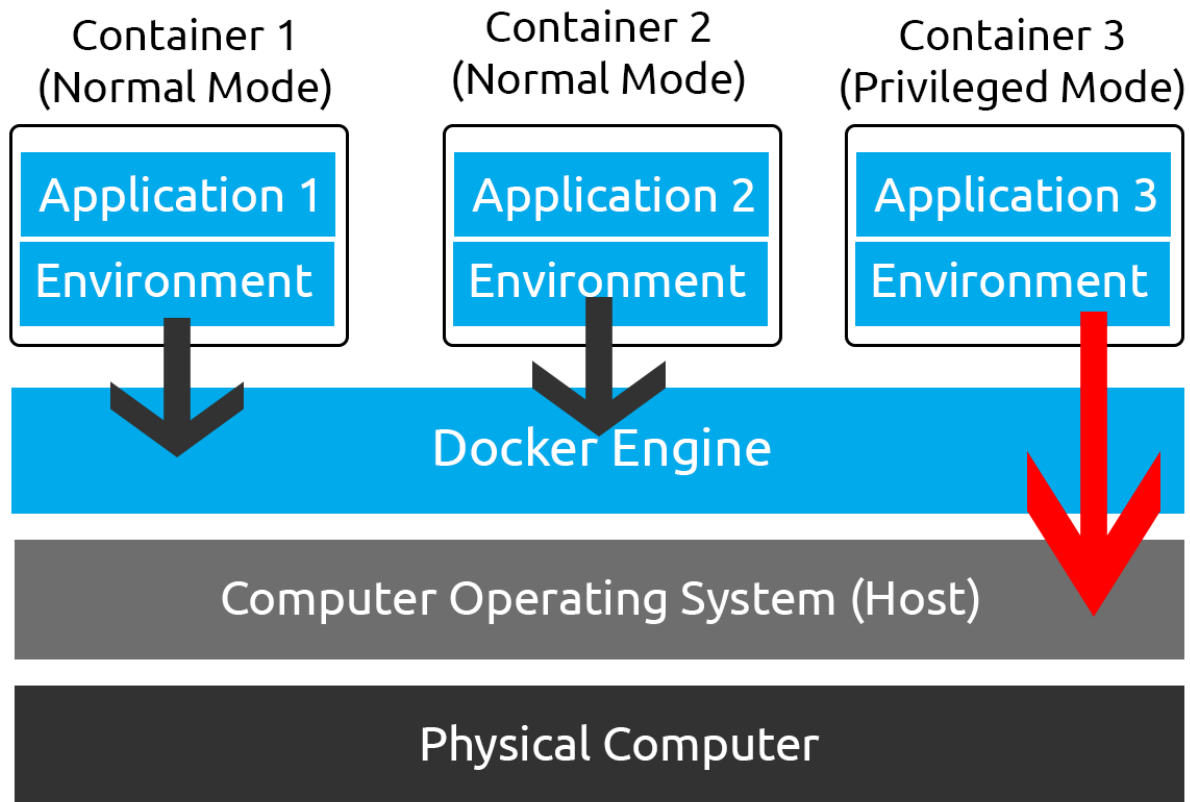
What would the command be if we wanted to inspect a docker container named "Apache"?

docker inspect apache

Task 4 - Preventing "Over-Privileged" Containers

Privileged containers bypass standard isolation mechanisms to gain unchecked access to the host, granting all capabilities and allowing full access to host resources, making them extremely dangerous despite rare legitimate uses like Docker-in-Docker or deep debugging.

A diagram demonstrating the difference of container mode privileges



Linux capabilities break down traditional all-or-nothing root privileges into granular permissions, allowing system administrators to grant processes only the specific system-level actions they require. I have placed some standard capabilities in the table below, what privileges they translate to, and where they may be used:

Capability	Description	Use Case
CAP_NET_BIND_SERVICE	This capability allows services to bind to ports, specifically those under 1024, which usually requires root privileges.	Allowing a web server to bind on port 80 without root access.
CAP_SYS_ADMIN	This capability provides a variety of administrative privileges, including being	You may find this capability in a process that automates administrative tasks. For

	able to mount/unmount file systems, changing network settings, performing system reboots, shutdowns, and more.	example, modifying a user or starting/stopping a service.
CAP_SYS_RESOURCE	This capability allows a process to modify the maximum limit of resources available. For example, a process can use more memory or bandwidth.	This capability can control the number of resources a process can consume on a granular level. This can be either increasing the amount of resources or reducing the amount of resources.

It's recommended assigning capabilities to containers individually rather than running containers with the `--privileged` flag (which will assign all capabilities). For example, you can assign the `NET_BIND_SERVICE` capability to a container running a web server on port 80 by including the `--cap-add=NET_BIND_SERVICE` when running the container. Assigning the `NET_BIND_SERVICE` capability to a container

```
cmnatic@thm:~# docker run -it --rm --cap-drop=ALL --cap-add=NET_BIND_SERVICE mywebserver
```

Finally, the command `capsh --print` can be used to determine what capabilities are assigned to a process. Using `capsh` to list the capabilities currently assigned

```
cmnatic@thm:~# capsh --print
Current: =
Bounding set = cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner
-- cut for brevity ---
cap_sys_rawio,cap_sys_chroot,cap_sys_ptrace,cap_sys_pacct,cap_sys_admin,
cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_sys_time,cap_sys_tty_config,cap_mknod
-- cut for brevity ---
Ambient set =
Current IAB:
```

```
Securebits: 00/0x0/1'b0
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
secure-no-ambient-raise: no (unlocked)
uid=1000(cmna1ic) euid=1000(cmna1ic)
gid=1000(cmna1ic)
groups=4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),1000
(cmna1ic)
```

It is important to frequently review what capabilities are assigned to a container. When a container is privileged, it shares the same namespace as the host, meaning resources on the host can be accessed by the container - breaking the "isolated" environment.

Answer the questions below

What is the name of the capability that allows services to bind to ports (specifically those under 1024)?

CAP_NET_BIND_SERVICE

What argument would we provide when starting a Docker container to add a capability?

-cap-add

Finally, what command (with argument) would we use to print the capabilities assigned to a process?

capsh --print

Task 5 - Seccomp & AppArmor 101

Seccomp (Secure Computing Mode) is a Linux kernel security feature that acts like a guard, enforcing rules to ensure applications only perform authorized actions. By defining profiles that restrict allowed system calls (such as allowing file reading while blocking network connections for reverse shells), Seccomp

limits an attacker's ability to execute malicious commands without impairing core application functionality.

```
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "architectures": [
    "SCMP_ARCH_X86_64",
    "SCMP_ARCH_X86",
    "SCMP_ARCH_X32"
  ],
  "syscalls": [
    { "names": [ "read", "write", "exit", "exit_group", "open", "close", "stat", "fstat", "lstat", "poll", "getdents", "munmap", "mprotect", "brk", "arch_prctl", "set_tid_address", "set_robust_list" ], "action": "SCMP_ACT_ALLOW" },
    { "names": [ "execve", "execveat" ], "action": "SCMP_ACT_ERRNO" }
  ]
}
```

This Seccomp profile:

- Allows files to be read and written to
- Allows a network socket to be created
- But does not allow execution (for example, `execve`)

To create a Seccomp profile, you can simply create a profile using your favourite text editor. This room will use `nano` . An example Seccomp profile (profile.json) has been provided below. This profile will allow reading and writing access to files but no network connections.

```
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "architectures": [ "SCMP_ARCH_X86_64" ],
  "syscalls": [
    {
```

```
    "name": "socket",
    "action": "SCMP_ACT_ERRNO",
    "args": []
  },
  {
    "name": "connect",
    "action": "SCMP_ACT_ERRNO",
    "args": []
  },
  {
    "name": "bind",
    "action": "SCMP_ACT_ERRNO",
    "args": []
  },
  {
    "name": "listen",
    "action": "SCMP_ACT_ERRNO",
    "args": []
  },
  {
    "name": "accept",
    "action": "SCMP_ACT_ERRNO",
    "args": []
  }
  {
    "name": "read",
    "action": "SCMP_ACT_ALLOW",
    "args": []
  },
  {
    "name": "write",
    "action": "SCMP_ACT_ALLOW",
    "args": []
  }
}
```

```
]
}
```

With our Seccomp profile now created, we can apply it to our container at runtime by using the `--security-opt seccomp` flag with the location of the Seccomp profile. For example: Applying our Seccomp profile when running a container

```
cmnatic@thm:~# docker run --rm -it --security-opt seccomp=/home/cmna
tic/container1/seccomp/profile.json mycontainer
```

AppArmor: AppArmor is an operating system-level Mandatory Access Control (MAC) feature that restricts unauthorized application actions based on system-wide rule profiles once installed and enabled on the host. To use AppArmor, we first need to ensure that it is installed on our system: Checking if AppArmor is installed or not

```
cmnatic@thm:~# sudo aa-status
apparmor module is loaded.
34 profiles are loaded.
```

With the output "apparmor module is loaded", we can confirm that AppArmor is installed and enabled. To apply an AppArmor profile to our container, we need to do the following:

- Create an AppArmor profile
- Load the profile into AppArmor
- Run our container with the new profile

First, let's create our AppArmor profile. You can use your favourite text editor for this. Note that there are tools out there that can help generate AppArmor profiles based on your Dockerfile. However, this is out-of-scope for this room and can be "unreliable". Provided below is an example AppArmor profile (profile.json) for an "Apache" web server that:

- Can read files located in `/var/www/`, `/etc/apache2/mime.types` and `/run/apache2`.

- Read & write to /var/log/apache2.
- Bind to a TCP socket for port 80 but not other ports or protocols such as UDP.
- Cannot read from directories such as /bin, /lib, /usr.

```
/usr/sbin/httpd {  
  
    capability setgid,  
    capability setuid,  
  
    /var/www/** r,  
    /var/log/apache2/** rw,  
    /etc/apache2/mime.types r,  
  
    /run/apache2/apache2.pid rw,  
    /run/apache2/*.sock rw,  
  
    # Network access  
    network tcp,  
  
    # System logging  
    /dev/log w,  
  
    # Allow CGI execution  
    /usr/bin/perl ix,  
  
    # Deny access to everything else  
    /** ix,  
    deny /bin/**,  
    deny /lib/**,  
    deny /usr/**,  
    deny /sbin/**  
}
```

Now that we have created the AppArmor profile, we will need to import this into the AppArmor program to be recognised. Importing our AppArmor profile into

AppArmor

```
cmnatic@thm:~# sudo apparmor_parser -r -W /home/cmnatic/container1/apparmor/profile.json
```

With our AppArmor profile now imported, we can apply it to our container at runtime by using the `--security-opt apparmor` flag with the location of the AppArmor profile. For example: Applying our AppArmor profile when running a container

```
cmnatic@thm:~# docker run --rm -it --security-opt apparmor=/home/cmnatic/container1/apparmor/profile.json mycontainer
```

What's the Difference: Well, to put it briefly:

- AppArmor determines what resources an application can access (i.e., CPU, RAM, Network interface, filesystem, etc) and what actions it can take on those resources.
- Seccomp is within the program itself, which restricts what system calls the process can make (i.e. what parts of the CPU and operating system functions).

It's important to note that it is not a "one or the other" case. Seccomp and AppArmor can be combined to create layers of security for a container.

Answer the questions below

If we wanted to enforce the container to only be able to read files located in /home/tryhackme, what type of profile would we use? Seccomp or AppArmor?

AppArmor

If we wanted to disallow the container from a system call (such as clock_adjtime), what type of profile would we use? Seccomp or AppArmor?

Seccomp

Finally, what command would we use if we wanted to list the status of AppArmor?

aa-status

Task 6 - Reviewing Docker Images

Reviewing Docker images prior to execution is critical because untrusted images —such as cryptomining malware discovered on Docker Hub with millions of pulls —pose severe risks, making image layer analysis via published Dockerfiles essential for security.

The screenshot shows the Docker Hub interface for the `nginx:latest` image. The top navigation bar includes the Docker Hub logo, a search bar with `nginx` entered, and links to Explore, Repositories, Organizations, and Help. A blue banner below the navigation bar shows the breadcrumb `Explore > library/nginx > latest`.

The main content area displays the `nginx:latest` image details. It includes the NGINX logo, a DIGEST hash, and a table of image metadata:

OS/ARCH	COMPRESSED SIZE	LAST PUSHED	TYPE	VULNERABILITIES
linux/amd64	54.25 MB	4 days ago by dojanky	Image	0 H 0 M 41 L

Below the metadata table, there is a section for "Image hierarchy" and "Layers (15)". The "Image hierarchy" section shows a tree view with `FROM debian:11-slim, 11.6-slim, bullseye-20230320-slim, bullseye-20230320-slim` and `ALL nginx:latest`. The "Layers (15)" section shows a list of layers with their IDs, commands, and sizes. The first layer is `ADD file:60911afdacfdc216e44115ad5f3cc07f41...` (31.41 MB). The second layer is `CMD ["bash"]` (0 B). The third layer is `LABEL maintainer=NGINX Docker Maintainers <dock...` (0 B).

On the right side of the "Image hierarchy" section, there is a tabbed interface with tabs for "Images (2)", "Vulnerabilities (41)", and "Packages (216)". The "Vulnerabilities (41)" tab is selected, showing a table of vulnerabilities:

Package	Vulnerabilities
debian/tiff 4.2.0-1+deb11u4	0 H 0 M 5 L
debian/pcrc3 2:8.39-13	0 H 0 M 4 L

In the image above, we can see the various layers of the image on DockerHub. These layers are the steps that are executed during the building process of the image. Additionally, open-source code repositories for images on the Docker Hub will often be included, allowing you to review the entire Dockerfile.

Code Blame 46 lines (34 loc) · 1.14 KB

```
1 FROM ubuntu:18.04
2
3 LABEL maintainer "@CMNatic <https://cmnatic.co.uk>"
4
5 RUN apt-get -qq update && \
6     DEBIAN_FRONTEND=noninteractive \
7     apt-get -y --no-install-recommends -qq install \
8     openssh-server \
9     apt-utils \
10    sudo=1.8.21p2-3ubuntu1 && \
11    mkdir -p /var/run/ssh && \
12    mkdir -p /root/.ssh && \
13    echo "Packages installed"
14
15 # Lets put this user add on another layer for clarity
16 RUN useradd -m uogctf -s /bin/bash && \
17     mkdir -p /home/uogctf/.ssh && \
18     echo "uogctf:uogctf" | chpasswd && \
19     echo "uogctf user successfully added"
20
21 # Add user to sudoers
22
23 RUN echo "uogctf ALL=(ALL,!root) NOPASSWD: /bin/bash" >> /etc/sudoers && \
```

In the image above, we can see the code for the Dockerfile. This allows us to audit the code and understand precisely what actions are being executed in the container. By analysing the code, we can check for vulnerabilities or malicious actions. Tools such as [Dive](#) (opens in new tab) allow you to reverse engineer Docker images by inspecting what is executed and changed at each layer of the image during the build process.

Task 7 - Compliance & Benchmarking

Compliance and benchmarking play vital roles in securing assets - let alone containers. Let us begin by explaining compliance. Compliance is the process of following regulations and standards such as the NIST SP 800-190, a set of standards from the National Institute of Standards and Technology that gives guidance and best practices on container security:

Compliance Framework	Description	URL
----------------------	-------------	-----

NIST SP 800-190	This framework outlines the potential security concerns associated with containers and provides recommendations for addressing these concerns.	https://csrc.nist.gov/publications/detail/sp/800-190/final (opens in new tab).
ISO 27001	This framework is an international standard for information security. The standard guides implementing, maintaining and improving an information security management system.	https://www.iso.org/standard/27001 (opens in new tab).

Please note that you may have to adhere to additional frameworks relevant to your Industry. For example, financial or medical. Regulations exist in all industries. For example, in the medical field, the HIPPA for handling medical data.

Benchmarking, on the other hand, is a process used to see how well an organisation is adhering to best practices. Benchmarking allows an organisation to see where they are following best practices well and where further improvements are needed:

Benchmarking Tool	Description	URL
CIS Docker Benchmark	This tool can assess a container's compliance with the CIS Docker Benchmark framework.	https://www.cisecurity.org/benchmark/docker (opens in new tab).
OpenSCAP	This tool can assess a	https://www.open-scap.org/ (opens in new tab).

	container's compliance with multiple frameworks, including CIS Docker Benchmark, NIST SP-800-190 and more.	
Docker Scout	This tool is a cloud-based service provided by Docker itself that scans Docker images and libraries for vulnerabilities. This tool lists the vulnerabilities present and provides steps to resolve these.	https://docs.docker.com/scout/(opens in new tab).
Anchore	This tool can assess a container's compliance with multiple frameworks, including CIS Docker Benchmark, NIST SP-800-190 and more.	https://github.com/anchore/anchore-engine(opens in new tab).
Grype	This tool is a modern and fast vulnerability scanner for Docker images	https://github.com/anchore/grype(opens in new tab).

An example of using the Docker Scout tool to analyse a Docker image has been provided in the terminal below. Please note this will need to be [installed\(opens in new tab\)](#) beforehand. You can read the [Docker Scout\(opens in new tab\)](#) documentation to learn more. Using Docker Scout to scan our "nginx" image for vulnerabilities

```
cmnatic@thm:~# docker scout cves local://nginx:latest
✓ SBOM of image already cached, 215 packages indexed
x Detected 22 vulnerable packages with a total of 45 vulnerabilities
```

Overview

	Analyzed Image
Target	local://nginx:latest
digest	4df6f9ac5341
platform	linux/amd64
vulnerabilities	0C 1H 18M 28L
size	91 MB
packages	215

Packages and Vulnerabilities

```
0C 1H 1M 3L glibc 2.35-0ubuntu3.1
pkg:deb/ubuntu/glibc@2.35-0ubuntu3.1?os_distro=jammy&os_name=
ubuntu&os_version=22.04
```

```
x HIGH CVE-2023-4911
https://scout.docker.com/v/CVE-2023-4911
Affected range : <2.35-0ubuntu3.4
Fixed version : 2.35-0ubuntu3.4
CVSS Score : 7.8
CVSS Vector : CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
```

```
x MEDIUM CVE-2023-5156
https://scout.docker.com/v/CVE-2023-5156
Affected range : <2.35-0ubuntu3.5
```

Fixed version : 2.35-0ubuntu3.5
CVSS Score : 7.5
CVSS Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/
I:N/A:H

x LOW CVE-2016-20013
<https://scout.docker.com/v/CVE-2016-20013>
Affected range : >=0
Fixed version : not fixed
CVSS Score : 7.5
CVSS Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/
I:N/A:H

x LOW CVE-2023-4813
<https://scout.docker.com/v/CVE-2023-4813>
Affected range : <2.35-0ubuntu3.5
Fixed version : 2.35-0ubuntu3.5
CVSS Score : 5.9
CVSS Vector : CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/
I:N/A:H

x LOW CVE-2023-4806
<https://scout.docker.com/v/CVE-2023-4806>
Affected range : <2.35-0ubuntu3.5
Fixed version : 2.35-0ubuntu3.5
CVSS Score : 5.9
CVSS Vector : CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/
I:N/A:H

Answer the questions below

What is the name of the framework published by the National Institute of Standards and Technology?

NIST SP 800-190

What is the name of the analysis tool provided by Docker?

Docker Scout

Task 8 - Practical

Your task is to use the [Grype](#) (opens in new tab), vulnerability scanner on the machine to analyse some Docker images. Grype can be used to analyse Docker images and container filesystems. You can refer to the table below as a cheat sheet to answer the questions in this task.

Example	Description	Command
Scanning a Docker image	Scan a Docker image for vulnerabilities.	<code>grype imagename --scope all-layers</code>
Scanning an exported container filesystem	Scan an exported container filesystem (i.e. from <code>docker image save</code>).	<code>grype /path/to/image.tar</code>

Please note, that for this room, you can safely ignore the "Unable to check for vulnerability database update" warning messages.

Answer the questions below

Use Docker to list the running containers on the system. What is the name of the container that is currently running?

couchdb

```
root@thm-container-hardening:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
0f02452fdc7c   vulhub/couchdb:3.2.1               "/docker-entrypoint..." 2 years ago    Up 9 minutes  0.0.0.0:4369->4369/tcp, :::4369->4369/tcp, 0.0.0.0:5984->5984/tcp, :::5984->5984/tcp, 0.0.0.0:9100->9100/tcp, :::9100->9100/tcp  couchdb
root@thm-container-hardening:~# grype struts2 --scope all-layers
✓ Loaded image
✓ Parsed image
✓ Cataloged contents
  ✓ Packages [186 packages]
  ✓ File digests [4,486 files]
  ✓ File metadata [4,486 locations]
✓ Scanned for vulnerabilities [91 vulnerability matches]
  by severity: 1 critical, 0 high, 28 medium, 46 low, 16 negligible
  by status: 18 fixed, 73 not-fixed, 0 ignored
[0030] ERROR failed to fetch latest version: Get "https://toolbox-data.anchore.io/grype/releases/latest/VERSION": dial
NAME          INSTALLED    FIXED-IN    TYPE    VULNERABILITY
```

Use Grype to analyse the "struts2" image. What is the name of the library marked as "Critical"?

struts2-core

Low	openssl	3.0.2-0ubuntu1.12	3.0.2-0ubuntu1.14	deb	CVE-2023-6129
Low	openssl	3.0.2-0ubuntu1.12	3.0.2-0ubuntu1.14	deb	CVE-2023-5678
Low	passwd	1:4.8.1-2ubuntu2.1		deb	CVE-2023-4641
Low	passwd	1:4.8.1-2ubuntu2.1		deb	CVE-2023-29383
Low	struts2-core	6.3.0.1	6.3.0.2	java-archive	GHSA-2j39-qcjm-42
Critical	wget	1.21.2-2ubuntu1		deb	CVE-2021-31879
Medium					

Use Grype to analyse the exported container filesystem located at /root/container.tar. What severity is the "CVE-2023-45853" rated as?

Critical

```

root@thm-container-hardening:~# grype /root/container.tar
✓ Parsed image sha256:62148d38223f1ea2cb216e9684e6aa16b13f2fa9df6af99368d099cb5c20b8d3
✓ Cataloged contents 42cd89c67c3c82c4021745eccae7c0df3507024065217689f49868f4facf4fde
├── ✓ Packages [291 packages]
├── ✓ File digests [13,845 files]
├── ✓ File metadata [13,845 locations]
✓ Scanned for vulnerabilities [841 vulnerability matches]
├── by severity: 38 critical, 185 high, 310 medium, 28 low, 267 negligible (13 unknown)
├── by status: 368 fixed, 473 not-fixed, 0 ignored
[0030] ERROR failed to fetch latest version: Get "https://toolbox-data.anchore.io/grype/releases/latest/VERSION": dial
NAME          INSTALLED      FIXED-IN      TYPE    VULNERABILITY SEVERITY
apt            2.2.4          (won't fix)   deb     CVE-2011-3374  Negligible
bash          5.1.2+deb11u1 (won't fix)   deb     CVE-2022-3715  High
bsdutils      1:2.36.1-8+deb11u1 (won't fix)   deb     CVE-2022-0563  Negligible
coreutils     8.32-4+b1      (won't fix)   deb     CVE-2016-2781  Low
coreutils     8.32-4+b1      (won't fix)   deb     CVE-2017-18018 Negligible
dbus          1.12.24-0+deb11u1 1.12.28-0+deb11u1 deb     CVE-2023-34969 Medium
dbus-user-session 1.12.24-0+deb11u1 1.12.28-0+deb11u1 deb     CVE-2023-34969 Medium
e2fsprogs     1.46.2-2       (won't fix)   deb     CVE-2022-1304  High
erlang-asn1   1:23.2.6+dfsg-1 1:23.2.6+dfsg-1+deb11u1 deb     CVE-2022-37026 Critical
erlang-asn1   1:23.2.6+dfsg-1 (won't fix)   deb     CVE-2022-37026 Critical

```